



МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«МИРЭА – Российский технологический университет»  
РТУ МИРЭА

---

## Практическая работа № 3

по дисциплине «Технологии разработки приложений на базе фреймворков»

Тема работы: Объектно-ориентированное программирование на Python

Выполнил: студент группы ЭФБО-11-24  
(группа)

Мишуков В.Р.  
(подпись) (Фамилия И.О.)

Принял: доц.  
(должность)

Ящун Т.В.  
(подпись) (Фамилия И.О.)

Москва  
2026

## 1. Индивидуальный проект

- Название проекта: Сервис регистрации продавцов на ярмарку (FairVendor), тема № 433.
- Перечень классов и функционала, реализованных на этапе ПРЗ (ООП):
  1. `models/fairs.py` (класс `Fair`): представление ярмарочного мероприятия. Атрибуты: `id`, `name`, `location`, `date`, `base_rate`, `total_space`. Методы: `is_space_available()` (проверка вместимости стенда), `get_free_space()` (расчет остатка площади), `__str__()`, `to_dict()`, `from_dict()`, а также функция поиска `find_fair_by_id()`;
  2. `models/vendors.py` (иерархия классов продавцов): базовый класс `Vendor` с инкапсулированной валидацией реквизитов (ID, наименование, 10/12-значный ИНН, документы, стаж). Специализированные подклассы с полиморфным переопределением тарифного коэффициента `get_category_coefficient()` и строкового вывода: `CraftVendor` (льготный тариф 1.0), `FoodVendor` (повышенный тариф 1.3), `SouvenirVendor` (тариф 1.1). Фабричный метод `Vendor.from_dict()`, генератор `filter_vendors_by_category()`, сортировка через `lambda` в `sort_vendors()`;
  3. `models/applications.py` (класс `Application`): связывает продавца и мероприятие через прямые объектные ссылки: `app.vendor: Vendor` и `app.fair: Fair`. Атрибуты: `requested_space`, `fee`, `status`, `is_cancelled`. Методы: `calculate_fee()` (вычисление стоимости на основе базовой ставки ярмарки, площади и полиморфного коэффициента продавца со скидкой 15% новичкам), `cancel()` (инкапсулированный отзыв брони), `generate_card()` (официальная карточка), `get_occupied_space()`, `create_application()`, `calculate_fair_statistics()`;
  4. `storage.py`: объектная сериализация/десериализация JSON (`load_fairs`, `save_fairs`, `load_vendors`, `save_vendors`, `load_applications`, `save_applications`) с автоматическим восстановлением связей объектов по идентификаторам и обработкой исключений;
  5. `main.py`: консольный интерфейс на 10 пунктов меню и автоматический демонстрационный сценарий (`--demo`).
- Структура модулей проекта: архитектура реорганизована в модульный пакет `models/`:
  - `models/__init__.py` — централизованный экспорт доменных моделей (`Fair`, `Vendor`, `Application`);
  - `models/fairs.py` — доменная модель ярмарочных площадок;
  - `models/vendors.py` — иерархия классов продавцов с полиморфизмом тарифов;
  - `models/applications.py` — объектная модель заявок и аналитика площадки;
  - `storage.py` — персистентное JSON-хранилище со связыванием объектов;
  - `main.py`, `fairs.py`, `vendors.py`, `applications.py`, `utils.py` — интерфейсный слой и слой обратной совместимости;
  - `data/` — каталог с файлами постоянного хранения (`fairs.json`, `vendors.json`, `applications.json`);
  - `tests/` — каталог модульных тестов (`test_fairs.py`, `test_vendors.py`, `test_applications.py`).
- Формат хранения данных: персистентное хранение организовано в формате JSON в каталоге `data/`. В файлах сохраняются нормализованные идентификаторы (`vendor_id`, `fair_id`), а при загрузке функция `load_applications()` автоматически восстанавливает объектную модель, связывая каждую заявку с соответствующими объектами `Vendor` и `Fair`.
- Ссылка на Git-репозиторий: <https://github.com/aplaygs/IP.git>

## 2. Результат работы

В результате выполнения практической работы № 3 (ПР3):

- осуществлен переход от процедурно-коллекционной модели (ПР2) к полноценной объектно-ориентированной архитектуре (ООП): создан доменный пакет `models/` с классами `Fair`, `Vendor`, `Application`;
- реализован принцип инкапсуляции: валидация реквизитов участников, площадей и ставок вынесена в конструкторы `__init__`, а управление состоянием заявки реализовано через защищенные методы (`app.cancel()`);
- построена иерархия наследования: от базового класса `Vendor` унаследованы специализированные классы `CraftVendor`, `FoodVendor` и `SouvenirVendor`;
- продемонстрирован динамический полиморфизм: каждый подкласс переопределяет метод `get_category_coefficient()` (ставки 1.0, 1.3, 1.1) и строковое представление `__str__()`, обеспечивая расчет сборов без ветвлений `if/elif`;
- реализована объектная ассоциация в классе `Application`, хранящем прямые ссылки на объекты `vendor`: `Vendor` и `fair`: `Fair`, что обеспечивает прозрачный доступ к свойствам связанных сущностей;
- разработан механизм сериализации и десериализации (`to_dict()`, `from_dict()`) с сохранением связей в JSON и автоматическим связыванием объектов при загрузке в `storage.py`;
- обеспечена полная обратная совместимость через методы `__getitem__` и `__contains__`, позволяющая бесшовно использовать существующие алгоритмы и тесты;
- реализован комплекс автоматизированных тестов `pytest` в каталоге `tests/` (модули `test_fairs.py`, `test_vendors.py`, `test_applications.py`, `test_main.py`) — все 33 теста успешно пройдены;
- код проверен линтером `flake8`: строгое соблюдение PEP 8 (длина строк, структура импортов, форматирование), 0 замечаний;
- обновлена документация `README.md`, зафиксирован единый итоговый коммит «Практика 3» в репозитории GitHub.

## 3. Вывод

В ходе выполнения практической работы № 3 были глубоко изучены и успешно применены на практике ключевые концепции объектно-ориентированного программирования на языке Python: классы, объекты, инкапсуляция, наследование, динамический полиморфизм и ассоциация сущностей. Проект «Сервис регистрации продавцов на ярмарку» был успешно переведен с функциональной парадигмы на профессиональную объектную архитектуру. Выделение доменных моделей `Fair`, `Vendor` и `Application` существенно повысило модульность, расширяемость и сопровождаемость системы, исключило дублирование логики и обеспечило строгое соблюдение бизнес-правил. Созданный автоматизированный тестовый набор из 33 тестов и безупречное соответствие PEP 8 подтверждают высокое качество реализации и готовность проекта к интеграции с веб-фреймворком Django на следующем этапе (ПР4).

Москва  
2026